

Database

Operazioni CRUD

Come creare, leggere, aggiornare e cancellare dati con PHP e SQL.

Cosa significa CRUD

CRUD indica le quattro operazioni principali sui dati:

- Create: creare
- Read: leggere
- Update: aggiornare
- Delete: cancellare

In un sito reale le usi continuamente: aggiungere un prodotto, mostrare una lista, modificare un profilo, eliminare un commento.

Create: inserire dati

```
<?php
$stmt = $pdo->prepare(
    "INSERT INTO prodotti (nome, prezzo) VALUES (:nome, :prezzo)"
);

$stmt->execute([
    "nome" => "Quaderno",
    "prezzo" => 3.50,
]);
```

Read: leggere dati

```
<?php
$stmt = $pdo->query("SELECT id, nome, prezzo FROM prodotti");
$prodotti = $stmt->fetchAll();

foreach ($prodotti as $prodotto) {
    echo $prodotto["nome"] . "\n";
}
```

`fetchAll` legge tutte le righe trovate.

Update: modificare dati

```
<?php
$stmt = $pdo->prepare(
    "UPDATE prodotti SET prezzo = :prezzo WHERE id = :id"
);

$stmt->execute([
    "prezzo" => 4.00,
    "id" => 1,
]);
```

`WHERE` è fondamentale: senza, rischi di modificare tutte le righe.

Delete: cancellare dati

```
<?php
$stmt = $pdo->prepare("DELETE FROM prodotti WHERE id = :id");
$stmt->execute(["id" => 1]);
```

Anche qui `WHERE` decide cosa cancellare.

Collegamento con le pagine web

Di solito un form raccoglie dati, PHP li valida, poi una query preparata li salva o li modifica.

Ordine consigliato:

1. ricevi i dati
2. controlla che siano validi
3. prepara la query
4. esegui la query
5. mostra un messaggio chiaro

Una piccola pagina di lettura

Una pagina che mostra prodotti potrebbe essere così:

```
<?php
$stmt = $pdo->query("SELECT id, nome, prezzo FROM prodotti ORDER BY nome");
$prodotti = $stmt->fetchAll();
?>

<ul>
  <?php foreach ($prodotti as $prodotto): ?>
    <li>
      <?= htmlspecialchars($prodotto["nome"], ENT_QUOTES, 'UTF-8') ?>
      - <?= $prodotto["prezzo"] ?> euro
    </li>
  <?php endforeach; ?>
</ul>
```

PHP legge i dati. HTML li mostra. `htmlspecialchars` protegge il testo prima di inserirlo nella pagina.

Confermare prima di cancellare

La cancellazione è l'operazione più delicata. In un'interfaccia reale conviene chiedere conferma e usare POST, non un semplice link GET.

```
<form method="post" action="elimina.php">
  <input type="hidden" name="id" value="1">
  <button type="submit">Elimina</button>
</form>
```

Riepilogo

CRUD non è una funzione speciale di PHP. È un modo per ricordare le azioni fondamentali sui dati. In PHP le realizzi con form, validazione, PDO e query preparate.

Collegarsi a un database con PDO

Come connettere PHP a un database usando PDO.

Cosa è PDO

PDO è uno strumento di PHP per parlare con database diversi usando un'interfaccia comune.

Puoi pensarlo come un adattatore: il tuo codice PHP usa PDO, e PDO si occupa di comunicare con il database.

Dati di connessione

Per collegarti servono:

- host, cioè dove si trova il database
- nome del database
- utente
- password
- charset, cioè il modo in cui vengono gestiti i caratteri

Primo collegamento

```
<?php
$dsn = "mysql:host=localhost;dbname=negozio;charset=utf8mb4";
$utente = "root";
$password = "";

$pdo = new PDO($dsn, $utente, $password);
```

`$dsn` descrive il tipo di database e dove trovarlo. In questo esempio usiamo MySQL.

Gestire gli errori

Aggiungi opzioni per ricevere errori chiari.

```
<?php
$pdo = new PDO($dsn, $utente, $password, [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
]);
```

`ERRMODE_EXCEPTION` fa lanciare eccezioni quando qualcosa va storto. `FETCH_ASSOC` restituisce righe come array associativi.

Proteggere le credenziali

Non mettere password reali in file pubblici o condivisi. Nei progetti veri si usano variabili d'ambiente o file di configurazione non caricati online.

Verificare il collegamento

```
<?php
echo "Connessione riuscita";
```

Se il codice arriva a questa riga senza errori, PHP è riuscito a collegarsi.

Gestire un errore di connessione

Una connessione può fallire: password sbagliata, database spento, nome del database errato. Per questo conviene usare `try` e `catch`.

```
<?php
try {
    $pdo = new PDO($dsn, $utente, $password, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    ]);

    echo "Connessione riuscita";
} catch (PDOException $errore) {
    echo "Impossibile collegarsi al database";
}
```

Durante lo studio puoi leggere il messaggio tecnico dell'errore. In un sito pubblico, invece, mostra un messaggio generico e registra il dettaglio nei log.

Primo controllo con una query semplice

Dopo la connessione puoi provare una query piccola.

```
<?php
$stmt = $pdo->query("SELECT 1");
$resultato = $stmt->fetchColumn();

echo $resultato;
```

Se vedi **1**, PHP ha parlato con il database e ha ricevuto una risposta.

Errore comune: confondere PDO con SQL

PDO non sostituisce SQL. PDO è il mezzo con cui PHP invia istruzioni SQL al database e riceve risultati.

Query preparate

Come inviare query sicure al database evitando SQL injection.

Il problema

Quando una query contiene dati inseriti dall'utente, non devi incollarli direttamente nella stringa SQL.

Questo e pericoloso:

```
<?php
$sql = "SELECT * FROM utenti WHERE email = '$email'";
```

Se `$email` contiene testo malevolo, la query puo cambiare significato. Questo rischio si chiama **SQL injection**.

Prepared statement

Una query preparata separa la struttura della query dai valori.

```
<?php
$sql = "SELECT * FROM utenti WHERE email = :email";
$stmt = $pdo->prepare($sql);
$stmt->execute(["email" => $email]);

$utente = $stmt->fetch();
```

`:email` e un placeholder, cioe un segnaposto. Il valore vero arriva dopo, in modo controllato.

SELECT sicura

```
<?php
$email = $_POST["email"] ?? "";

$stmt = $pdo->prepare("SELECT * FROM utenti WHERE email = :email");
$stmt->execute(["email" => $email]);

$utente = $stmt->fetch();

if ($utente) {
    echo "Utente trovato";
}
```

`fetch` legge una riga. Se non ci sono risultati, restituisce `false` .

INSERT sicuro

```
<?php
$stmt = $pdo->prepare(
    "INSERT INTO utenti (nome, email) VALUES (:nome, :email)"
);

$stmt->execute([
    "nome" => $nome,
    "email" => $email,
]);
```

Anche qui i valori non vengono incollati nella query.

Regola pratica

Ogni volta che una query usa dati esterni, usa `prepare` ed `execute` con placeholder. E una delle abitudini più importanti per scrivere PHP sicuro con i database.

Placeholder con punto interrogativo

Oltre ai placeholder con nome, puoi usare `?` .

```
<?php
$stmt = $pdo->prepare("SELECT * FROM utenti WHERE email = ?");
$stmt->execute([$email]);
```

Funziona bene per query piccole. I placeholder con nome sono spesso più leggibili quando i valori sono molti.

Leggere più righe

Per una lista di risultati usa `fetchAll` .

```
<?php
$stmt = $pdo->prepare("SELECT * FROM prodotti WHERE prezzo < :massimo");
$stmt->execute(["massimo" => 20]);

$prodotti = $stmt->fetchAll();

foreach ($prodotti as $prodotto) {
    echo $prodotto["nome"] . "\n";
}
```

La query resta sicura perché il valore `20` viene passato separatamente.

Errore comune: preparare e poi incollare lo stesso

Questo non va bene:

```
<?php
$stmt = $pdo->prepare("SELECT * FROM utenti WHERE email = '$email'");
```

Anche se usi `prepare` , hai comunque incollato `$email` nella stringa. Il valore deve entrare tramite placeholder.